Formation Chef de Projet IA

Fondamentaux du Machine Learning

Thomas Walter Chloé-Agathe Azencott

Center for Computational Biology (CBIO)
Mines Paris – Institut Curie – INSERM U1331
PSL Research University & PR[AI]RIE, Paris, France

October 16, 2026

Programme

- Introduction à l'apprentissage automatique
- **Cours 1 :** Minimisation du risque empirique
- **Cours 2 :** Régularisation et sélection de modèle
- **Cours 3 :** Arbres et méthodes ensemblistes
- **Cours 4 :** Méthodes à noyaux
- **Cours 5 :** Réduction de dimension
- **Cours 6 :** Clustering
- Travaux pratiques : mise en œuvre des notions et algorithmes avec scikit-learn (Python numérique).

Cours 2 – Régularisation et sélection de modèle

0. Régression polynomiale

$\mathcal{D} = \{(\vec{x}^i, y^i)\}_{i=1,\dots,n} \qquad \vec{x}^i \in \mathbb{R}^p \quad y^i \in \mathbb{R}$

- Nous savons apprendre un modèle de la forme $(x_1, x_2, \dots, x_p) \mapsto \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \beta_p x_p$ **(régression linéaire)**
- Comment apprendre un modèle de la forme $(x_1, x_2, \dots, x_p) \mapsto \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \beta_p x_p + \beta_{11} x_1^2 + \beta_{12} x_1 x_2 + \dots + \beta_{pp} x_p^2$?

0. Régression polynomiale

$\mathcal{D} = \{(\vec{x}^i, y^i)\}_{i=1,\dots,n} \qquad \vec{x}^i \in \mathbb{R}^p \quad y^i \in \mathbb{R}$

- Nous savons apprendre un modèle de la forme $(x_1, x_2, \dots, x_p) \mapsto \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \beta_p x_p$ **(régression linéaire)**
- Comment apprendre un modèle de la forme $(x_1, x_2, \dots, x_p) \mapsto \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \beta_p x_p + \beta_{11} x_1^2 + \beta_{12} x_1 x_2 + \dots + \beta_{pp} x_p^2$?

  Nouvelle représentation : $\varphi : (x_1, x_2, \dots, x_p) \mapsto (1, x_1, x_2, \dots, x_p, x_1^2, x_1 x_2, \dots, x_p^2) \in \mathbb{R}^m \quad m = \frac{(p+1)(p+2)}{2}$

  Une fonction linéaire de $\varphi(\vec{x})$ est une fonction quadratique de $\vec{x}$ **(régression quadratique)**
- Se généralise à tout degré d en créant $\binom{p+d}{p}$ variables **(régression polynomiale)**

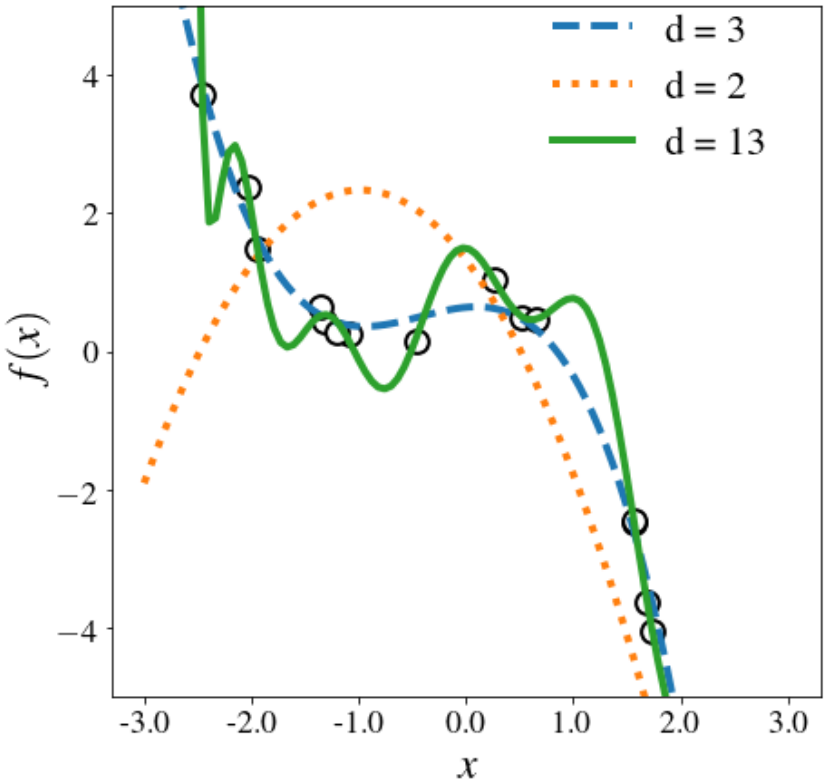
1. Généralisation et surapprentissage

- **Généralisation :** capacité du modèle à **faire de bonnes prédictions sur des données dont on ne connaît pas l'étiquette**
- **Défi principal** de l'apprentissage supervisé : le **surapprentissage** (overfitting)
  - i.e. la performance est meilleure sur les données d'apprentissage que sur de nouvelles données
  - **Plutôt facile :** entraîner un modèle qui prédise bien sur les données d'apprentissage
    Surtout avec **beaucoup de paramètres**
  - **Difficile :** garantir qu'il prédise bien sur de nouvelles données
    Exemple extrême : **apprentissage « par cœur »**

1. Généralisation et surapprentissage

- **données** $\mathcal{D} = \{(\vec{x}^i, y^i)\}_{i=1,\dots,n}$
- **hypothèses**
  - $(\vec{x}^i, y^i)$ réalisation de (X_i, Y_i) i.i.d et de même distribution que (X, Y)
  - $Y = \Phi(X) + \varepsilon$
- Le surapprentissage vient de la difficulté à ne pas capturer le **bruit** ε en approchant Φ
  - **bruit de mesure**
  - **bruit d'étiquetage**
  - **insuffisance du modèle**

1. Généralisation et surapprentissage

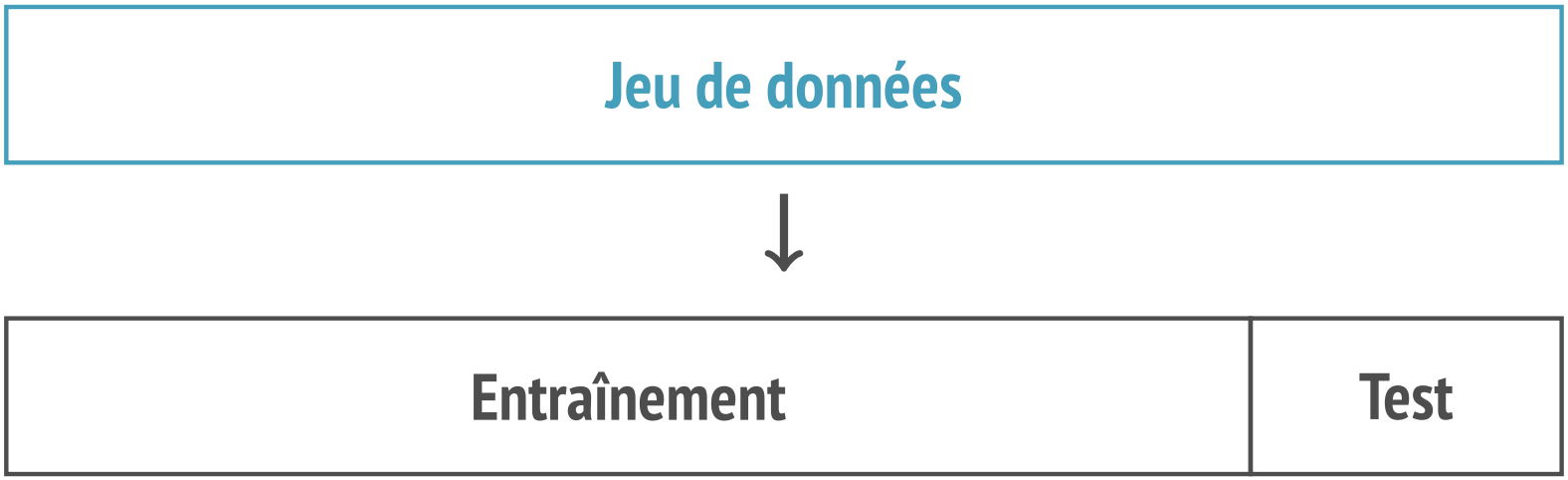


2. Erreur de généralisation

- **Erreur de généralisation** : erreur **sur de nouvelles données** du modèle $\hat{f}_{\mathcal{D}}$ appris sur $\mathcal{D}$
- Formellement : $\mathbb{E}[L(Y, \hat{f}_{\mathcal{D}}(X))]$
- En pratique, **estimée** en **mettant de côté** une partie des données :
  - On sépare les données en un **jeu d'entraînement** et un **jeu de test** (typiquement, 80%–20%)

2. Erreur de généralisation

- **Erreur de généralisation** : erreur **sur de nouvelles données** du modèle $\hat{f}_{\mathcal{D}}$ appris sur $\mathcal{D}$
- Formellement : $\mathbb{E}[L(Y, \hat{f}_{\mathcal{D}}(X))]$
- En pratique, **estimée** en **mettant de côté** une partie des données :
  - On sépare les données en un **jeu d'entraînement** et un **jeu de test** (typiquement, 80%–20%)



  - Le **jeu d'entraînement** (train set) sert à apprendre le modèle
  - Le **jeu de test** sert à estimer l'erreur de généralisation du modèle

Règle d'or

NE PAS TOUCHER au jeu de test

sauf pour évaluer l'erreur de généralisation du modèle

2. Erreur de généralisation

- La **validation croisée** (cross-validation) permet :
  - d'utiliser toutes les données pour l'entraînement et pour la validation
  - d'obtenir une performance moyenne (+/- écart-type) moins sensible au choix du jeu de test
- On sépare le jeu de données en K **blocs** (folds)

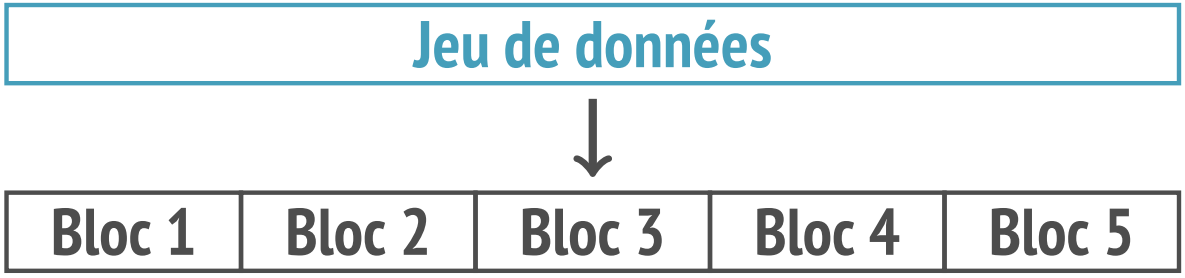
  En pratique, K=5 ou K=10 le plus souvent (équilibre entre le nombre d'expériences et la taille de chaque jeu d'entraînement)
- On utilise tour à tour chacun des blocs comme **jeu de validation** et l'union des autres comme **jeu d'entraînement**

$\Rightarrow$ K erreurs $\rightarrow$ **erreur de généralisation** du modèle

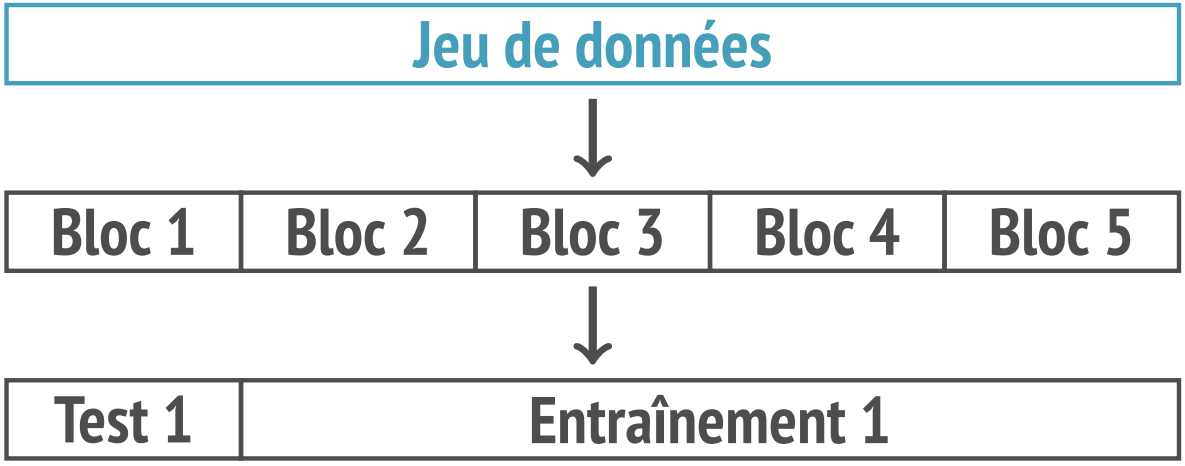
Validation croisée



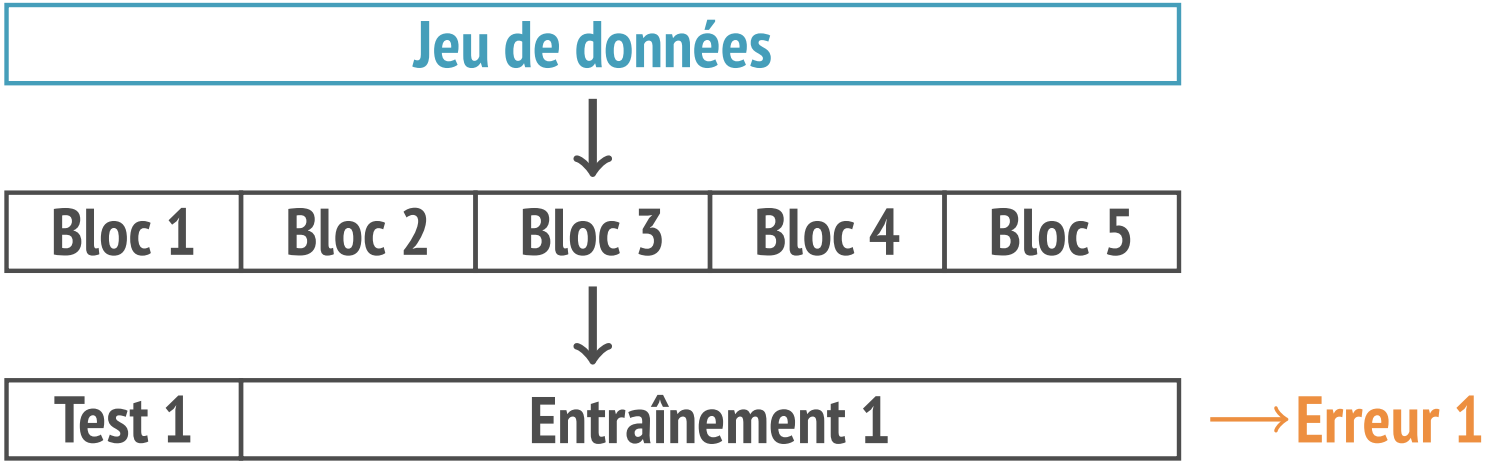
Validation croisée



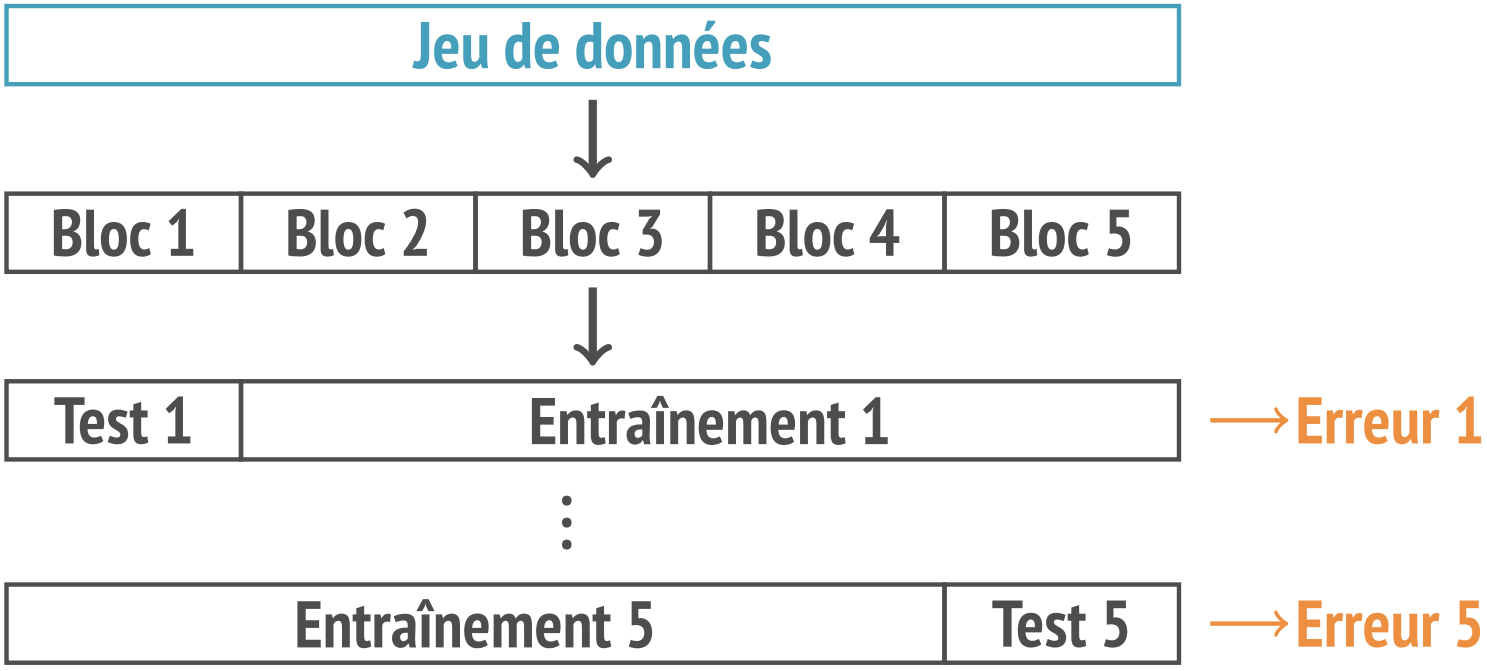
Validation croisée



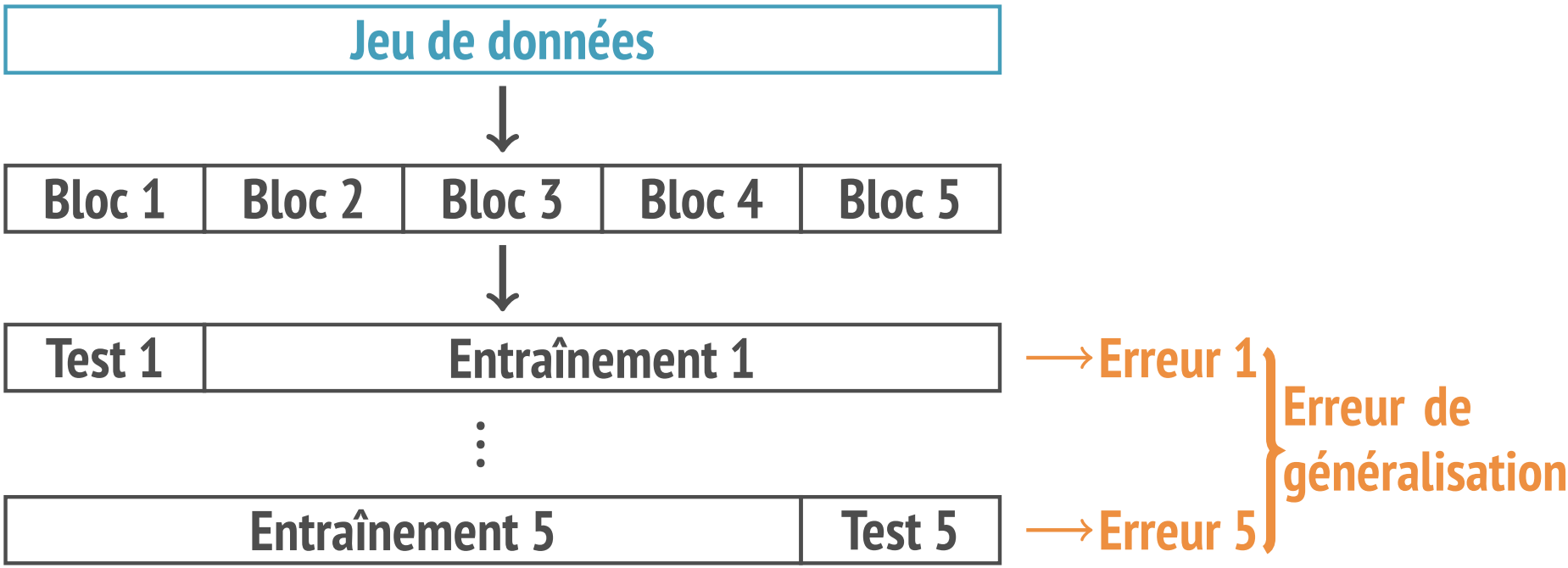
Validation croisée



Validation croisée



Validation croisée



L'erreur moyenne sur les K folds est une estimation de l'erreur de généralisation du modèle entraîné sur l'ensemble des données.

3. Critères de performance (classification)

- **Pourcentage d'erreur :** proportion d'observations mal classifiées

3. Critères de performance (classification)

- **Pourcentage d'erreur :** proportion d'observations mal classifiées
- **Problème :** cas où les classes ne sont **pas équilibrées**

  Exemple : détection de fraude
  - 99% des observations ne sont pas des fraudes
  - Un modèle qui prédit toujours « non » a un pourcentage d'erreur de 1%.

3. Critères de performance (classification)

- **Pourcentage d'erreur :** proportion d'observations mal classifiées
- **Problème :** cas où les classes ne sont **pas équilibrées**

  Exemple : détection de fraude
  - 99% des observations ne sont pas des fraudes
  - Un modèle qui prédit toujours « non » a un pourcentage d'erreur de 1%.
- **Matrice de confusion** (confusion matrix)

		Classe réelle	
		0	1
Classe prédite	0	Vrais Négatifs (TN)	Faux Négatifs (FN)
	1	Faux Positifs (FP)	Vrais Positifs (TP)

3. Critères de performance (classification)

- **Pourcentage d'erreur :** proportion d'observations mal classifiées
- **Problème :** cas où les classes ne sont **pas équilibrées**

  Exemple : détection de fraude
  - 99% des observations ne sont pas des fraudes
  - Un modèle qui prédit toujours « non » a un pourcentage d'erreur de 1%.
- **Matrice de confusion** (confusion matrix)

		True class	
		0	1
Predicted class	0	True Negatives (TN)	False Negatives (FN)
	1	False Positives (FP)	True Positives (TP)

3. Critères de performance (classification)

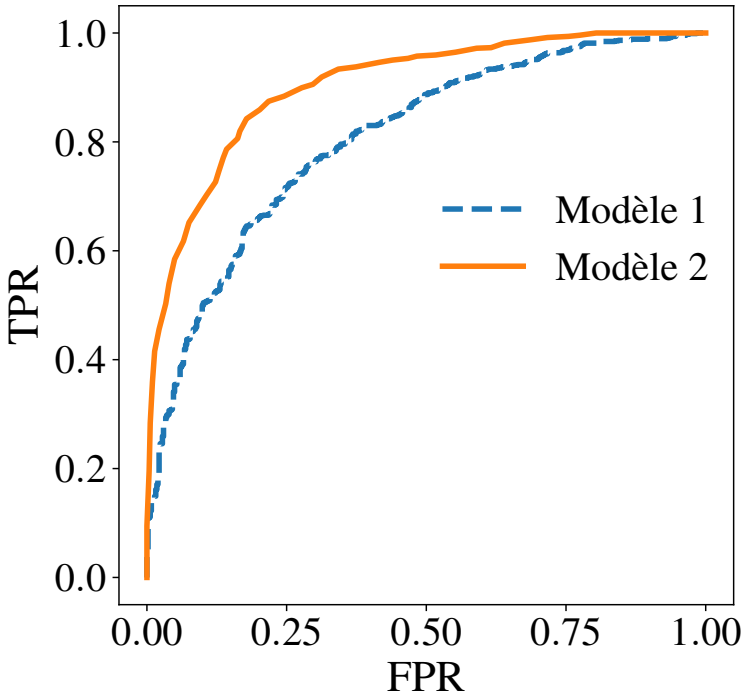
		Classe réelle		
		0	1	
Classe prédite	0	Vrais Négatifs (TN)	Faux Négatifs (FN)	TN+FN
	1	Faux Positifs (FP)	Vrais Positifs (TP)	TP+FP
		TN + FP	TP + FN	

- Sensibilité (sensitivity) = rappel (recall) = taux de vrais positifs = $\frac{TP}{TP + FN}$
- Spécificité (specificity) = taux de vrais négatifs (TN rate) = $\frac{TN}{TN + FP}$
- Précision (precision) = $\frac{TP}{TP + FP}$
- Précision (accuracy) = $\frac{TP + TN}{TP + FP + TN + FN}$
- Score F (F-score) = moyenne harmonique (précision, rappel) = $\frac{2\,TP}{TP + FP + TP + FN}$

3. Critères de performance (classification)

Courbe ROC (Receiver-Operator Characteristic)

- Évaluer la **fonction de décision** avant qu'elle ne soit seuillée
- **Choisir** le seuil de décision
- Résumé : **aire sous la courbe ROC** (AUC, AUROC)



TPR = True Positive Rate = $\frac{TP}{P}$ = sensibilité

FPR = False Positive Rate = $\frac{FP}{N}$ = 1 - spécificité

3. Évaluation d'un modèle (régression)

- **Compter** le nombre d'erreurs n'a pas de sens

3. Évaluation d'un modèle (régression)

- **Compter** le nombre d'erreurs n'a pas de sens
- Somme des carrés des erreurs : RSS $= \sum_{i=1}^{n} (y_i - f(\vec{x}_i))^2$
- Racine de l'erreur quadratique moyenne (Root Mean Squared Error) :

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - f(\vec{x}_i))^2}$$

- Corrélation de Pearson :

$$R^2 = \frac{\sum_{i=1}^{n} (y_i - \overline{y}) \left(f(\vec{x}_i) - \overline{f(\vec{x})} \right)}{\sqrt{\sum_{i=1}^{n} (y_i - \overline{y})^2} \sqrt{\sum_{i=1}^{n} \left(f(\vec{x}_i) - \overline{f(\vec{x})} \right)^2}}$$

4. Régularisation

- Par principe, la **minimisation du risque empirique** conduit facilement au **surapprentissage**
- La **régularisation** consiste à **contraindre** le problème de MRE pour **limiter la complexité** du modèle
  - **Complexité** $\Omega(f)$ d'un modèle f : notion théorique, par ex. Vapnik–Chervonenkis
  - Intuitivement : **flexibilité** des modèles que l'on peut apprendre
  - Proxys : nombre de paramètres ; amplitude possible de ces paramètres

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(\vec{x}_i)) + \lambda \, \Omega(f) \quad \lambda > 0$$

Coefficient de régularisation

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(\vec{x}_i)) + \lambda \, \Omega(f) \quad \lambda > 0$$

- **Coefficient de régularisation** $\lambda > 0$: gouverne l'importance relative entre la minimisation du risque empirique et la complexité $\Omega(f)$ du modèle.

- $\lambda = 0$:

- $\lambda \to +\infty$:

4. Régularisation

- Exemple 1 : **Régularisation ridge** (ou ℓ_2, ou **weight decay**)
  - Pour un modèle paramètrique de paramètres $\theta_1, \theta_2, \ldots, \theta_m$, $\Omega(\vec{\theta}) = \left|\left|\vec{\theta}\right|\right|_2^2 = \sum_{j=1}^{m} \theta_j^2$ contrôle **l'amplitude des paramètres**

4. Régularisation

- Exemple 1 : **Régularisation ridge** (ou ℓ_2, ou **weight decay**)
  - Pour un modèle paramètrique de paramètres $\theta_1, \theta_2, \ldots, \theta_m$, $\Omega(\vec{\theta}) = \left|\left|\vec{\theta}\right|\right|_2^2 = \sum_{j=1}^{m} \theta_j^2$ contrôle **l'amplitude des paramètres**
  - **Régression (linéaire) ridge :**

$$\underset{\vec{\beta} \in \mathbb{R}^{p+1}}{\arg\min} \frac{1}{n} \left(\vec{y} - X\vec{\beta}\right)^\top \left(\vec{y} - X\vec{\beta}\right) + \lambda \left|\left|\vec{\beta}\right|\right|_2^2$$

    Admet toujours une unique solution $\vec{\beta}^* = \left(X^\top X + \lambda I_p\right)^{-1} X^\top \vec{y}$ car ajouter une matrice diagonale à coefficients strictement positifs à $X^\top X$ la rend inversible

4. Régularisation

- Exemple 2 : **Régularisation** ℓ_1
  - Pour un modèle paramètrique de paramètres $\theta_1, \theta_2, \ldots, \theta_m$, $\Omega(\vec{\theta}) = \left|\left|\vec{\theta}\right|\right|_1 = \sum_{j=1}^{m} |\theta_j|$ contrôle **le nombre de paramètres non nuls** $\rightarrow$ **parcimonie** (sparsity)

4. Régularisation

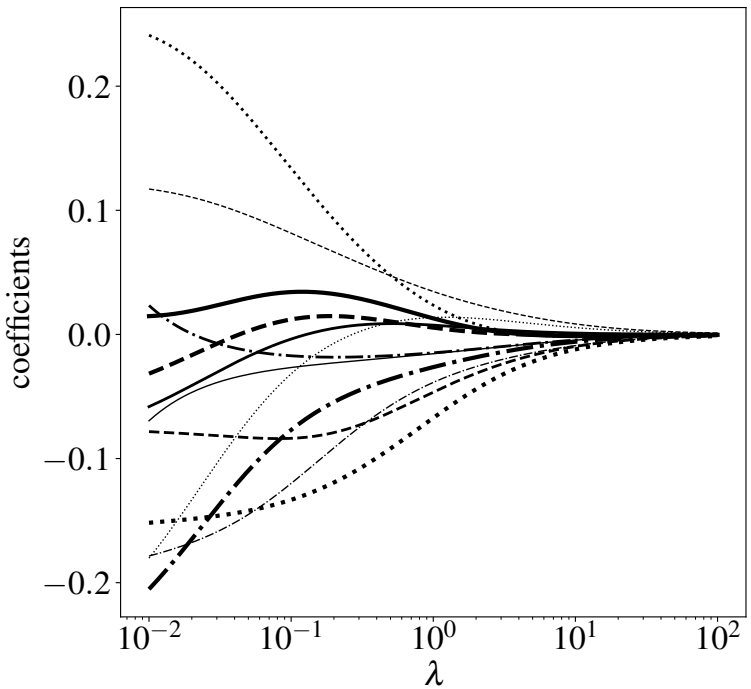
- Exemple 2 : **Régularisation** ℓ_1
  - Pour un modèle paramètrique de paramètres $\theta_1, \theta_2, \ldots, \theta_m$, $\Omega(\vec{\theta}) = \left|\left|\vec{\theta}\right|\right|_1 = \sum_{j=1}^{m} |\theta_j|$ contrôle **le nombre de paramètres non nuls** $\longrightarrow$ **parcimonie** (sparsity)
  - **Lasso** (Least Absolute Sparse Selection Operator) :

$$\underset{\vec{\beta} \in \mathbb{R}^{p+1}}{\arg\min} \frac{1}{n} \left(\vec{y} - X\vec{\beta}\right)^{\top} \left(\vec{y} - X\vec{\beta}\right) + \lambda \left|\left|\vec{\beta}\right|\right|_1$$

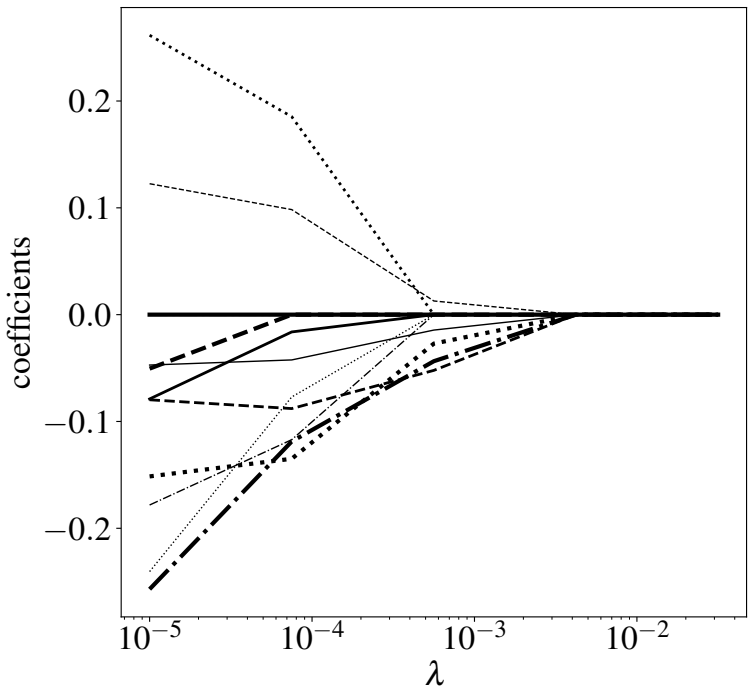
  - Pas de solution analytique

4. Régularisation

Chemins de régularisation : Chaque courbe correspond à une variable



Ridge



Lasso

4. Régularisation

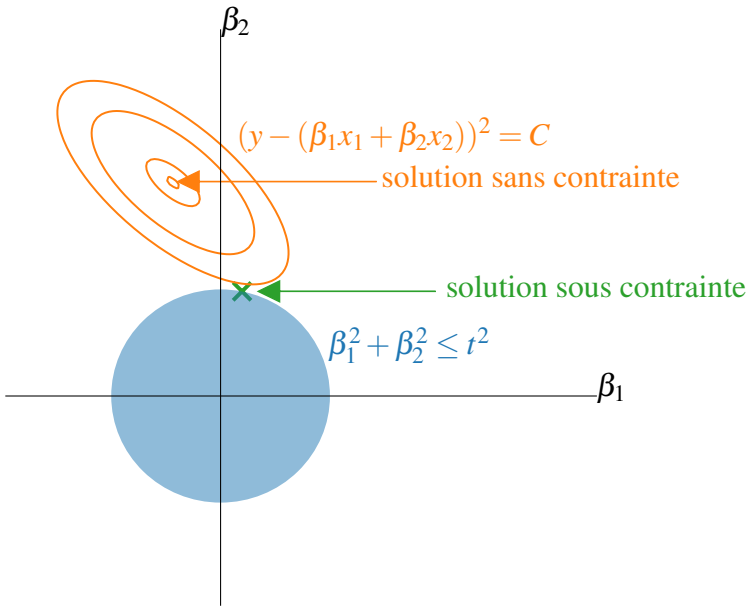
Point de vue du compromis biais-variance

$\mathcal{D} = \{(\vec{x}^i, y^i)\}_{i=1,\dots,n}$ $(\vec{x}^i, y^i)$ réalisation de (X_i, Y_i) copies i.i.d de (X, Y) $Y = \Phi(X) + \varepsilon$

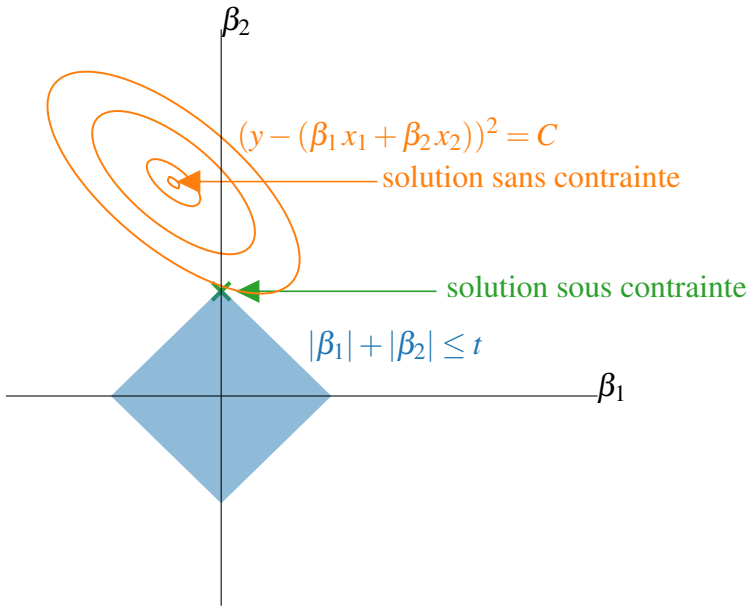
- Pour une observation $\vec{x}$: $f(\vec{x})$ est la réalisation de F estimateur de $\Phi(\vec{x})$
  F est une fonction de $(X_1, X_2, \dots, X_n, Y_1, Y_2, \dots Y_n)$
- **EQM** de F : $\mathbb{E}[(F - \Phi(\vec{x}))^2] = \text{Biais}(F) + \text{Var}(F)$ **(compromis biais-variance)**
- **Régularisation : réduire l'EQM** en **biaisant** le modèle de sorte à **réduire sa variance**
  (variance $\equiv$ complexité)

4. Régularisation

Point de vue géométrique



Régularisation ℓ_2



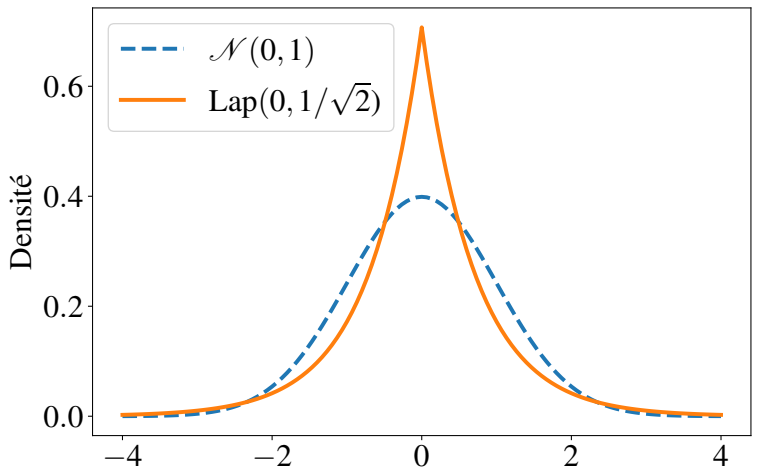
Régularisation ℓ_1

4. Régularisation

Point de vue de l'estimation statistique

Sous hypothèse de bruit gaussien $Y = \Phi(X) + \varepsilon \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2)$

- MRE $\equiv$ estimation des paramètres du modèle par maximum de vraisemblance
- **MRE + régularisation ℓ_2** $\equiv$ estimation des paramètres du modèle par maxium a posteriori, avec une prior **gaussienne**
- **MRE + régularisation ℓ_1** $\equiv$ estimation des paramètres du modèle par maxium a posteriori, avec une prior **de Laplace**



4. Régularisation

Elastic Net

- Variables **corrélées** : le Lasso sélectionne l'une d'entre elles aléatoirement

$\Rightarrow$ **instabilité**

- **Elastic Net**

$$\Omega(\vec{\theta}) = \alpha \left|\left|\vec{\theta}\right|\right|_2^2 + (1-\alpha) \left|\left|\vec{\theta}\right|\right|_1 \qquad 0 < \alpha < 1$$

5. Sélection de modèle

- **Hyperparamètre :** paramètre **de l'algorithme d'apprentissage** et non pas du modèle
  - Un hyperparamètre **définit** le problème de minimisation du risque empirique
  - Un paramètre est **déterminé** (= **appris**) par la résolution du problème de minimisation du risque empirique
- Ex :
  - **Coefficient de régularisation** λ
  - **Degré** d d'une régression polynomiale.

5. Sélection de modèle

Recherche sur grille (grid search)

– On cherche l'**hyperparamètre** qui donne **la meilleure performance** sur le jeu de **test.**

$$\text{Ex} : \widehat{\lambda} \in \arg\min_{\lambda>0} \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \text{Err}\left(y^i, \widehat{f}_\lambda(\vec{x}^i)\right) ;$$

$$\widehat{f}_\lambda \in \arg\min_{f\in\mathcal{F}} \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} L(y_i, f(\vec{x}_i)) \; + \; \lambda\, \Omega(f)$$

5. Sélection de modèle

Recherche sur grille (grid search)

- On cherche l'**hyperparamètre** qui donne **la meilleure performance** sur le jeu de **test.**

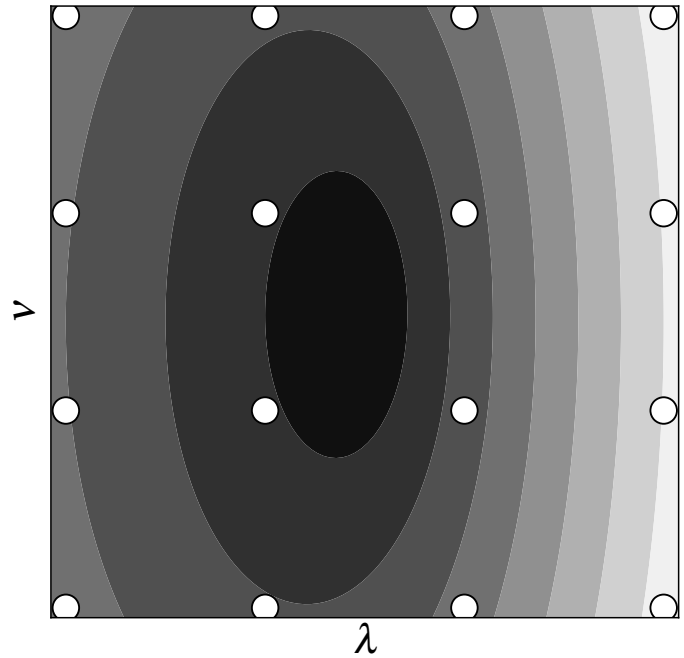
$$\text{Ex} : \widehat{\lambda} \in \arg\min_{\lambda>0} \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \text{Err}\left(y^i, \widehat{f}_\lambda(\vec{x}^i)\right) ;$$

$$\widehat{f}_\lambda \in \arg\min_{f\in\mathcal{F}} \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} L(y_i, f(\vec{x}_i)) \; + \; \lambda\, \Omega(f)$$

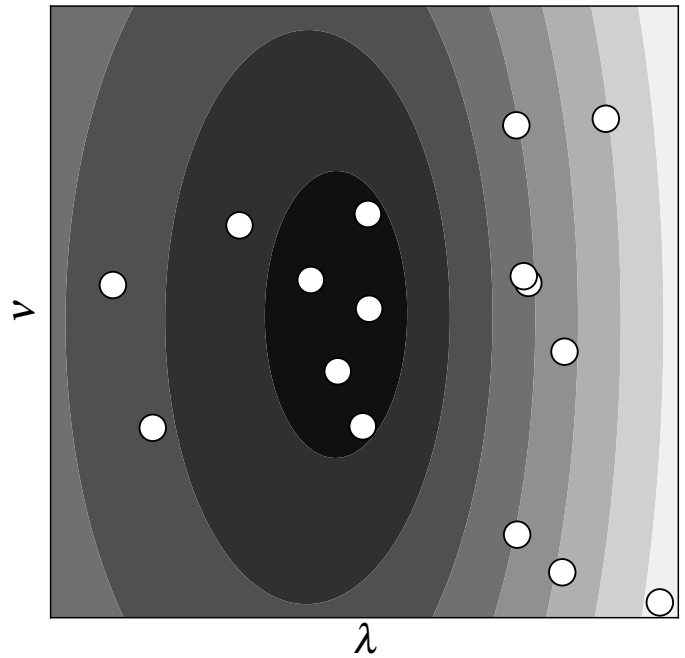
- En pratique :
  - On teste une **grille** (liste) prédéfinie de valeurs de l'hyperparamètre
    Ex : $\lambda \in [10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2, 10^3]$
  - Plus grande liberté sur le choix de Err que de L

5. Sélection de modèle

Plusieurs hyperparamètres



Recherche sur grille (grid search)



Recherche aléatoire (random search)

5. Sélection de modèle

- Pour déterminer **le meilleur modèle** parmi ceux appris :
  - avec différents algorithmes d'apprentissage ;
  - avec différentes valeurs d'hyperparamètre(s) pour le même algorithme
- **Idée :** sélectionner celui qui a la **meilleure performance sur le jeu de test.**

5. Sélection de modèle

- Pour déterminer **le meilleur modèle** parmi ceux appris :
  - avec différents algorithmes d'apprentissage ;
  - avec différentes valeurs d'hyperparamètre(s) pour le même algorithme
- **Idée :** sélectionner celui qui a la **meilleure performance sur le jeu de test.**
- **Attention :** on ne peut plus utiliser le jeu de test pour déterminer l'**erreur de généralisation !**

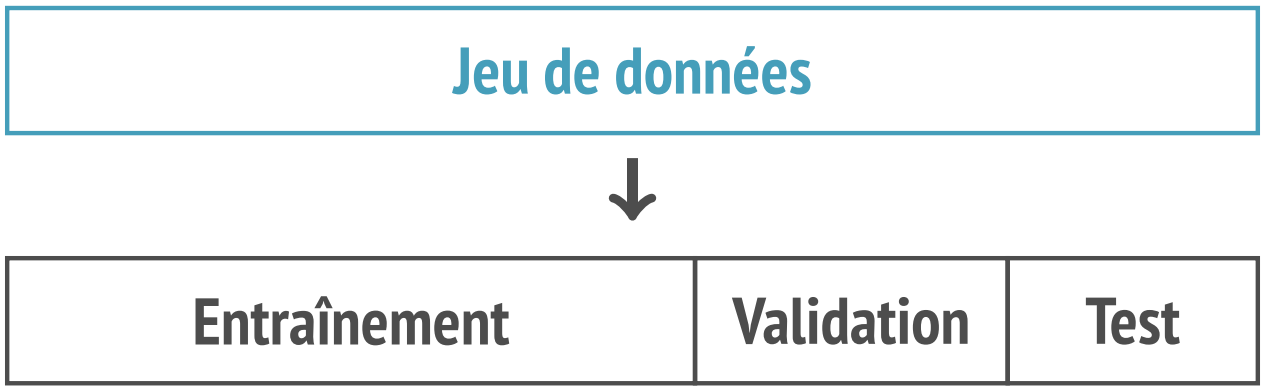
5. Sélection de modèle

- Pour déterminer **le meilleur modèle** parmi ceux appris :
  - avec différents algorithmes d'apprentissage ;
  - avec différentes valeurs d'hyperparamètre(s) pour le même algorithme
- **Idée :** sélectionner celui qui a la **meilleure performance sur le jeu de test.**
- **Attention :** on ne peut plus utiliser le jeu de test pour déterminer l'**erreur de généralisation !**
- On sépare les données en 3 jeux : apprentissage, **validation** et test.



5. Sélection de modèle

- Pour déterminer **le meilleur modèle** parmi ceux appris :
  - avec différents algorithmes d'apprentissage ;
  - avec différentes valeurs d'hyperparamètre(s) pour le même algorithme
- **Idée :** sélectionner celui qui a la **meilleure performance sur le jeu de test.**
- **Attention :** on ne peut plus utiliser le jeu de test pour déterminer l'**erreur de généralisation !**
- On sépare les données en 3 jeux : apprentissage, **validation** et test.



5. Sélection de modèle

Entraînement	Validation	Test

- Le **jeu d'entraînement** sert à apprendre **chacun des** modèles
- Le **jeu de validation** sert à **sélectionner** le meilleur modèle
- Le **jeu de test** sert à estimer l'erreur de généralisation **du modèle sélectionné**

5. Sélection de modèle

Entraînement	Validation	Test

- Le **jeu d'entraînement** sert à apprendre **chacun des** modèles
- Le **jeu de validation** sert à **sélectionner** le meilleur modèle
- Le **jeu de test** sert à estimer l'erreur de généralisation **du modèle sélectionné**
- **Problèmes :**
  - On utilise seulement une fraction des données pour l'entraînement
  - Possiblité que le modèle sélectionné ait une meilleure performance sur ce jeu de validation précis à cause d'un artefact.

5. Sélection de modèle

Validation croisée (sélection de modèle)

- On peut aussi utiliser la **validation croisée** pour la **sélection de modèle**
- On sépare le jeu d'entraînement en K **blocs** (ou folds)
- Pour chaque algorithme ou valeur d'hyperparamètre à évaluer :
  - On utilise tour à tour chacun des blocs comme **jeu de validation** et l'union des autres comme **jeu d'entraînement**

  $\Rightarrow$ on obtient K scores de performance $\rightarrow$ **performance moyenne**
- On sélectionne l'algorithme ou la valeur d'hyperparamètre qui donne la meilleure performance moyenne
- On réentraîne sur le jeu d'entraînement l'algorithme sélectionné
- On estime l'erreur de généralisation en évaluant sur le jeu de test le modèle ainsi obtenu

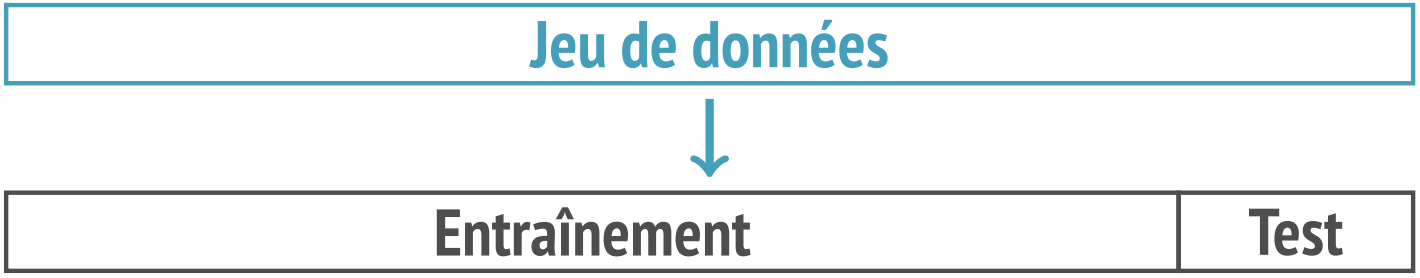
5. Sélection de modèle

Validation croisée (sélection de modèle)

Jeu de données

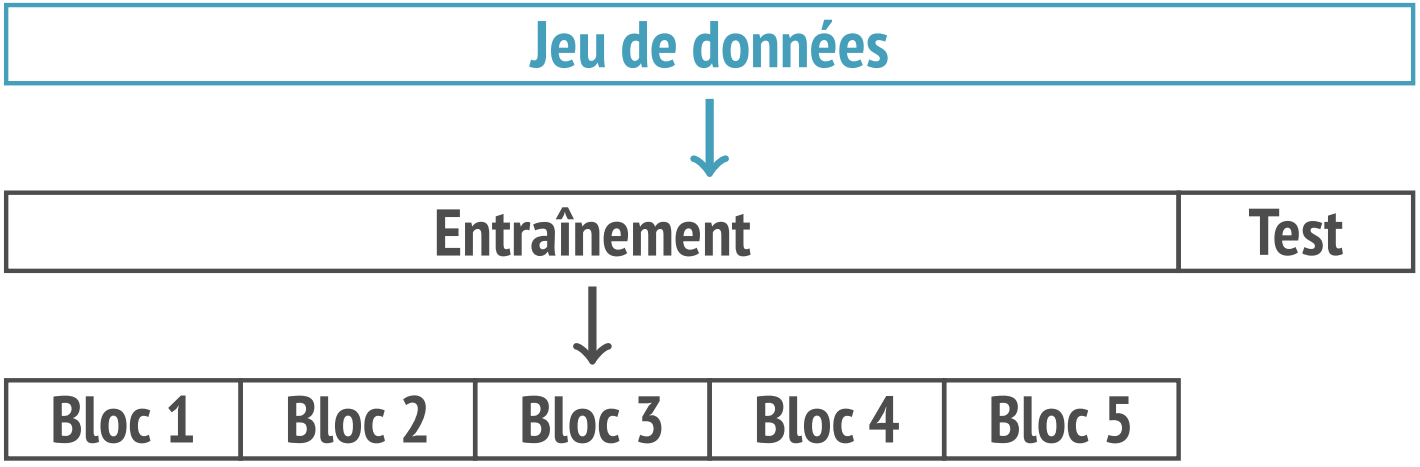
5. Sélection de modèle

Validation croisée (sélection de modèle)



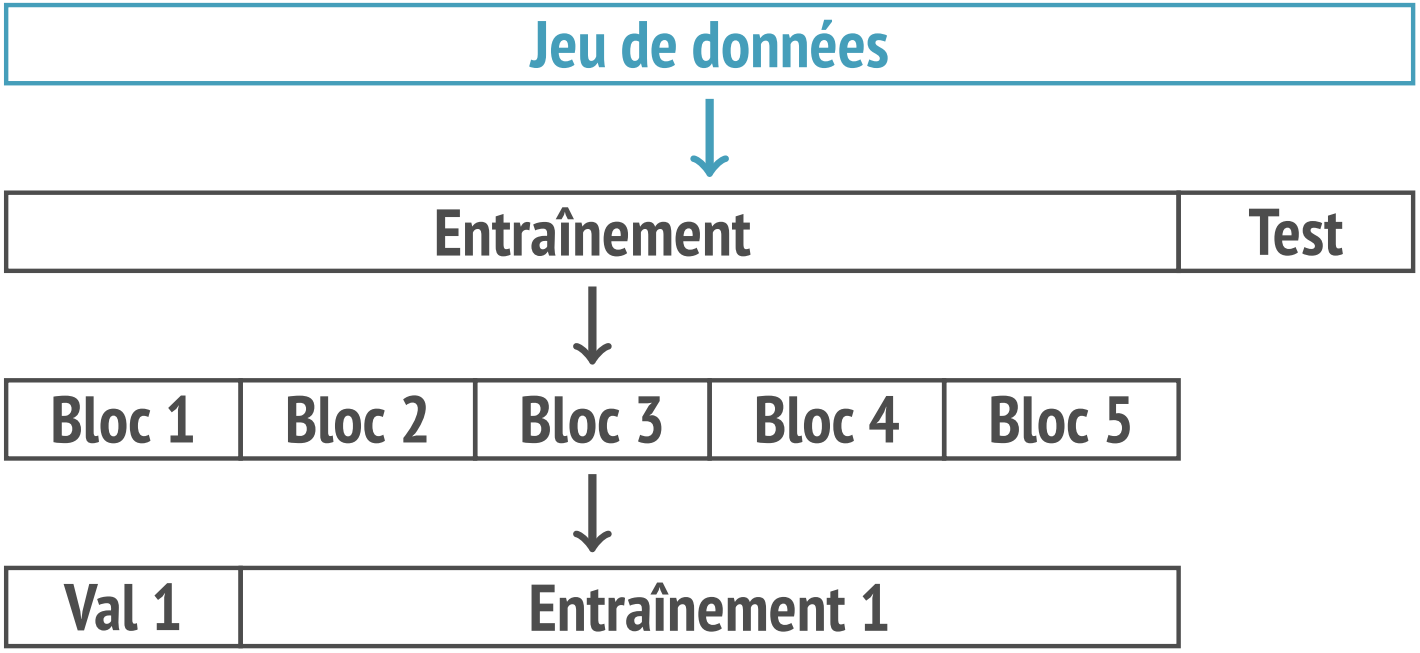
5. Sélection de modèle

Validation croisée (sélection de modèle)



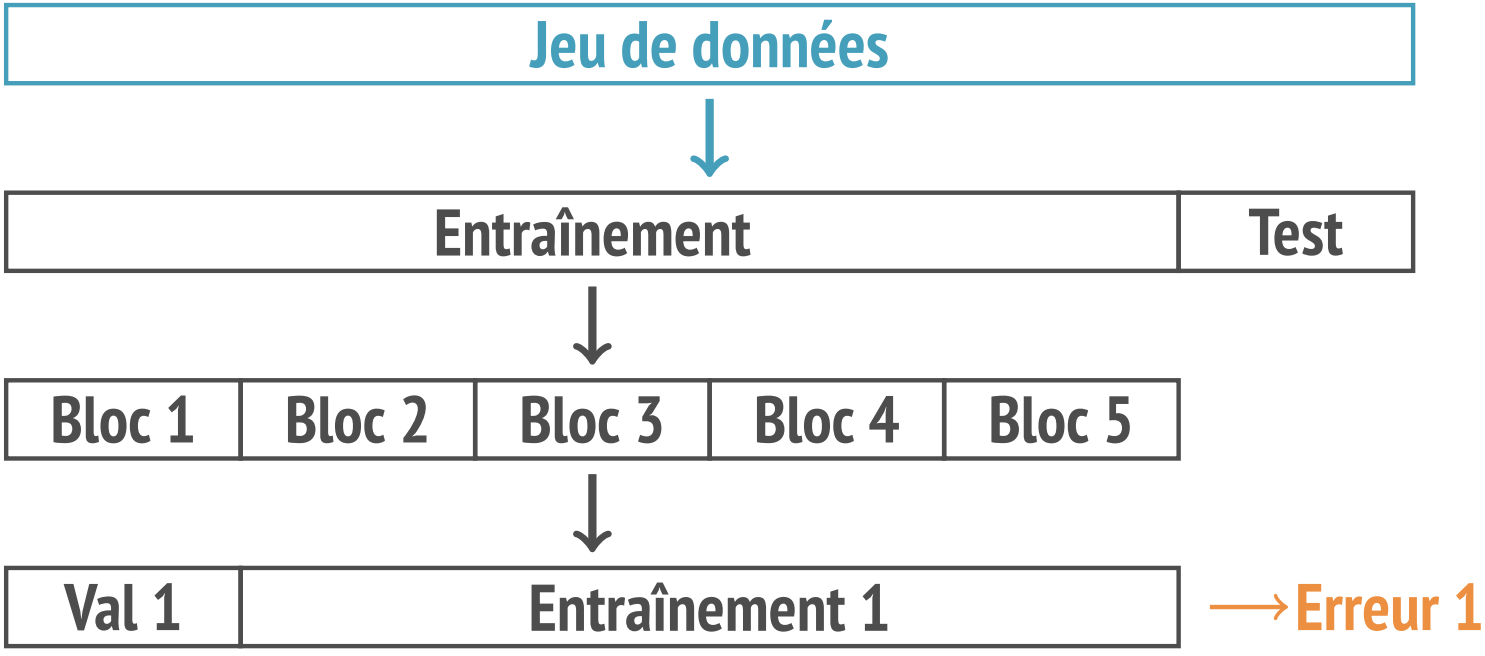
5. Sélection de modèle

Validation croisée (sélection de modèle)



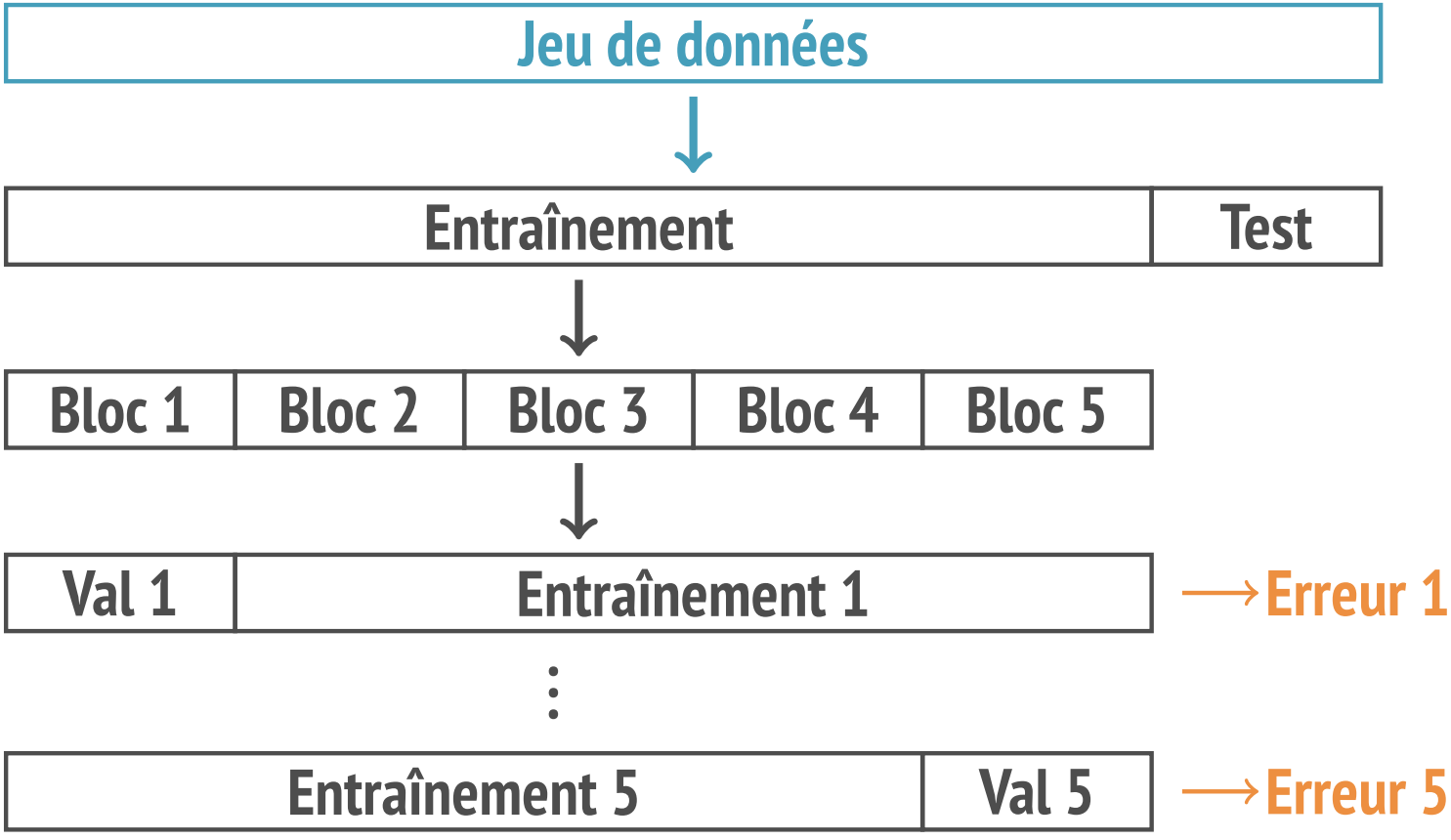
5. Sélection de modèle

Validation croisée (sélection de modèle)



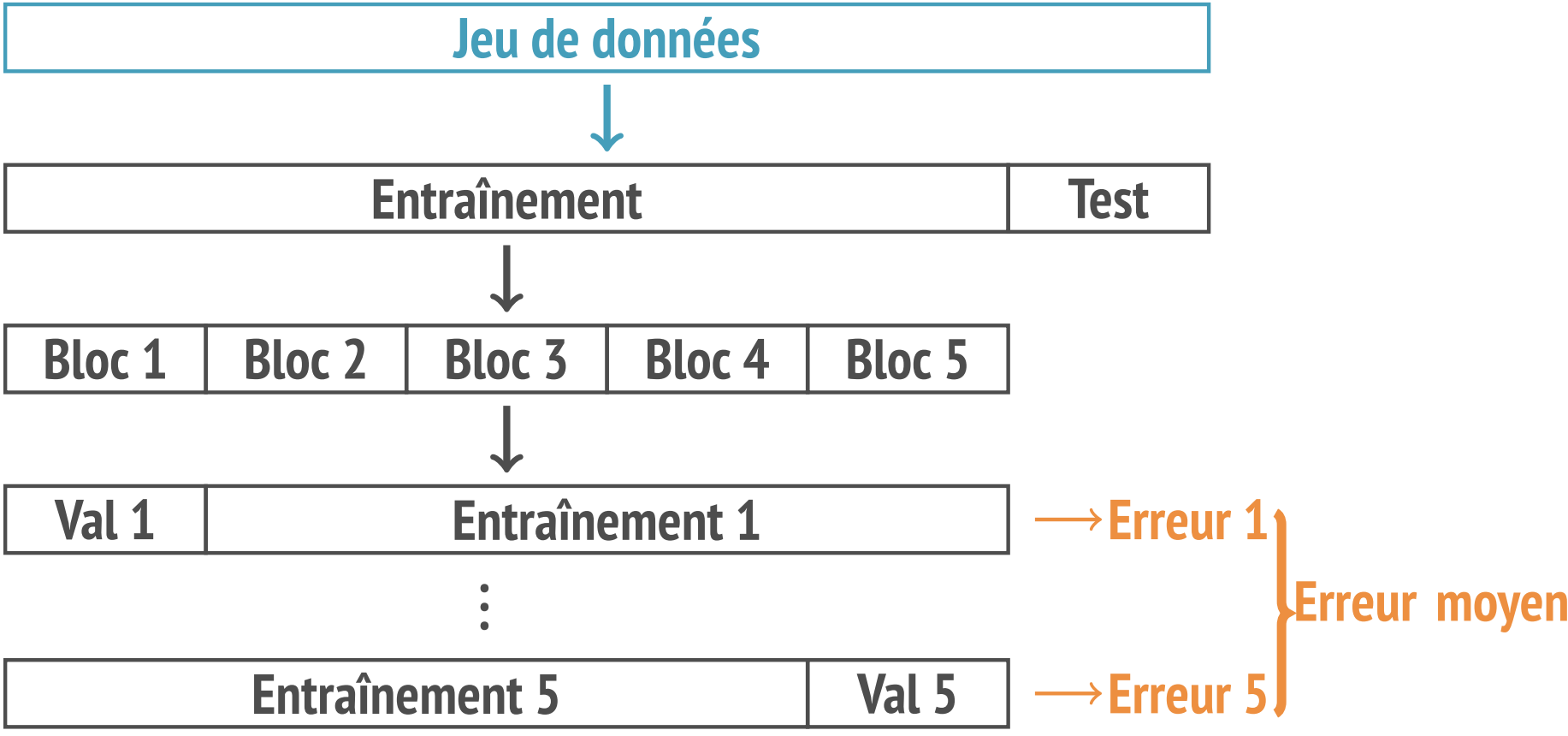
5. Sélection de modèle

Validation croisée (sélection de modèle)



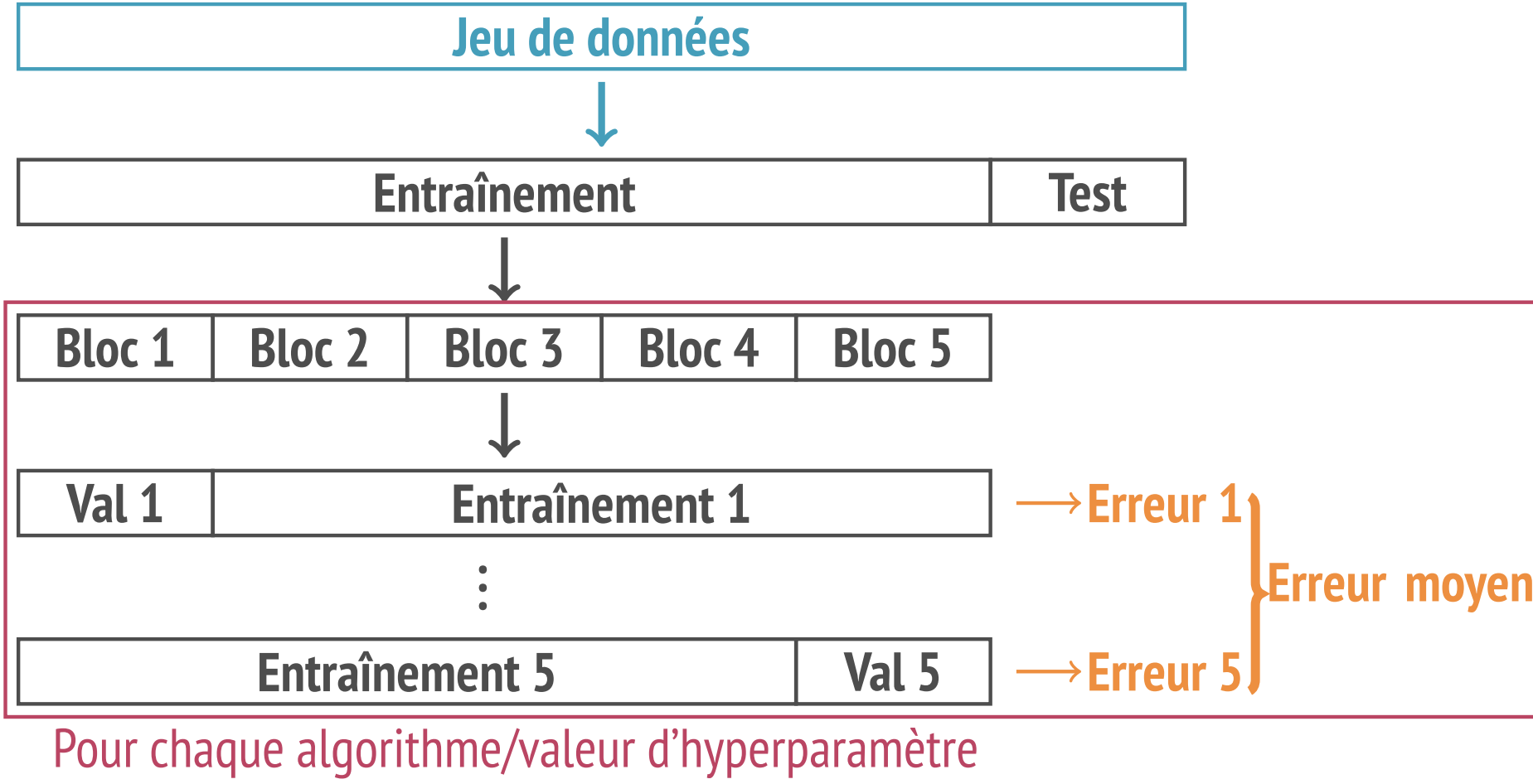
5. Sélection de modèle

Validation croisée (sélection de modèle)



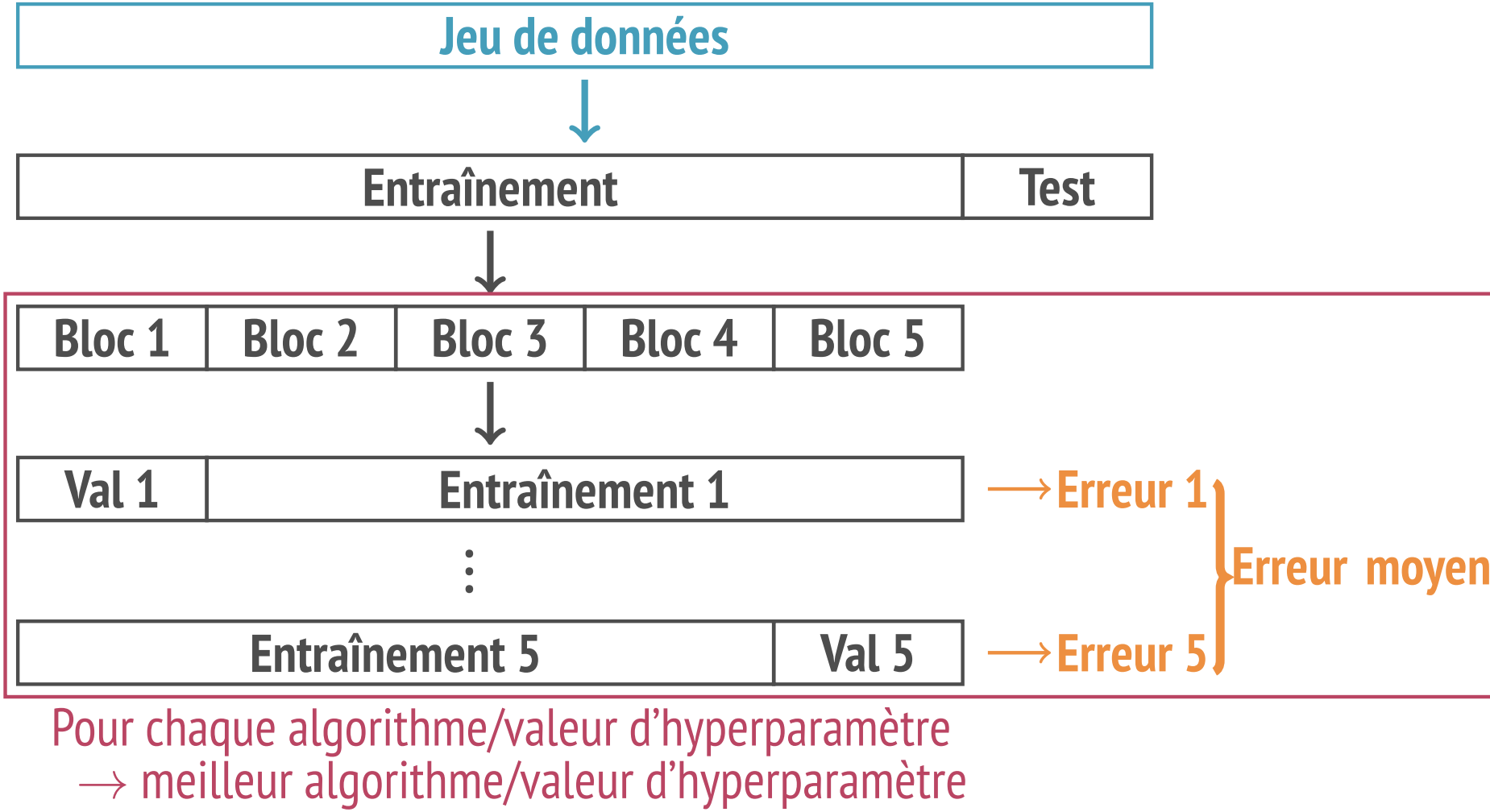
5. Sélection de modèle

Validation croisée (sélection de modèle)



5. Sélection de modèle

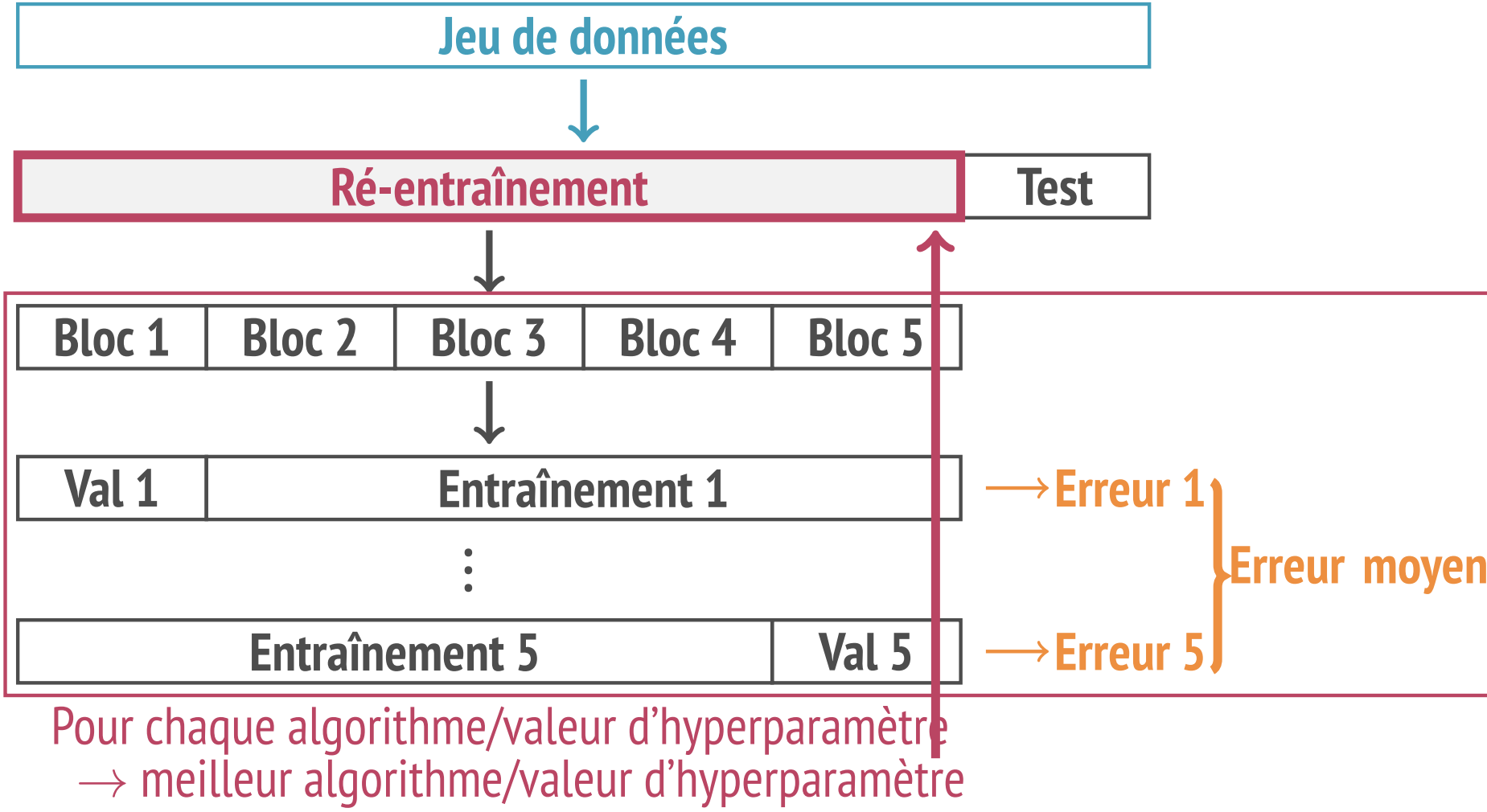
Validation croisée (sélection de modèle)



Pour chaque algorithme/valeur d'hyperparamètre
→ meilleur algorithme/valeur d'hyperparamètre

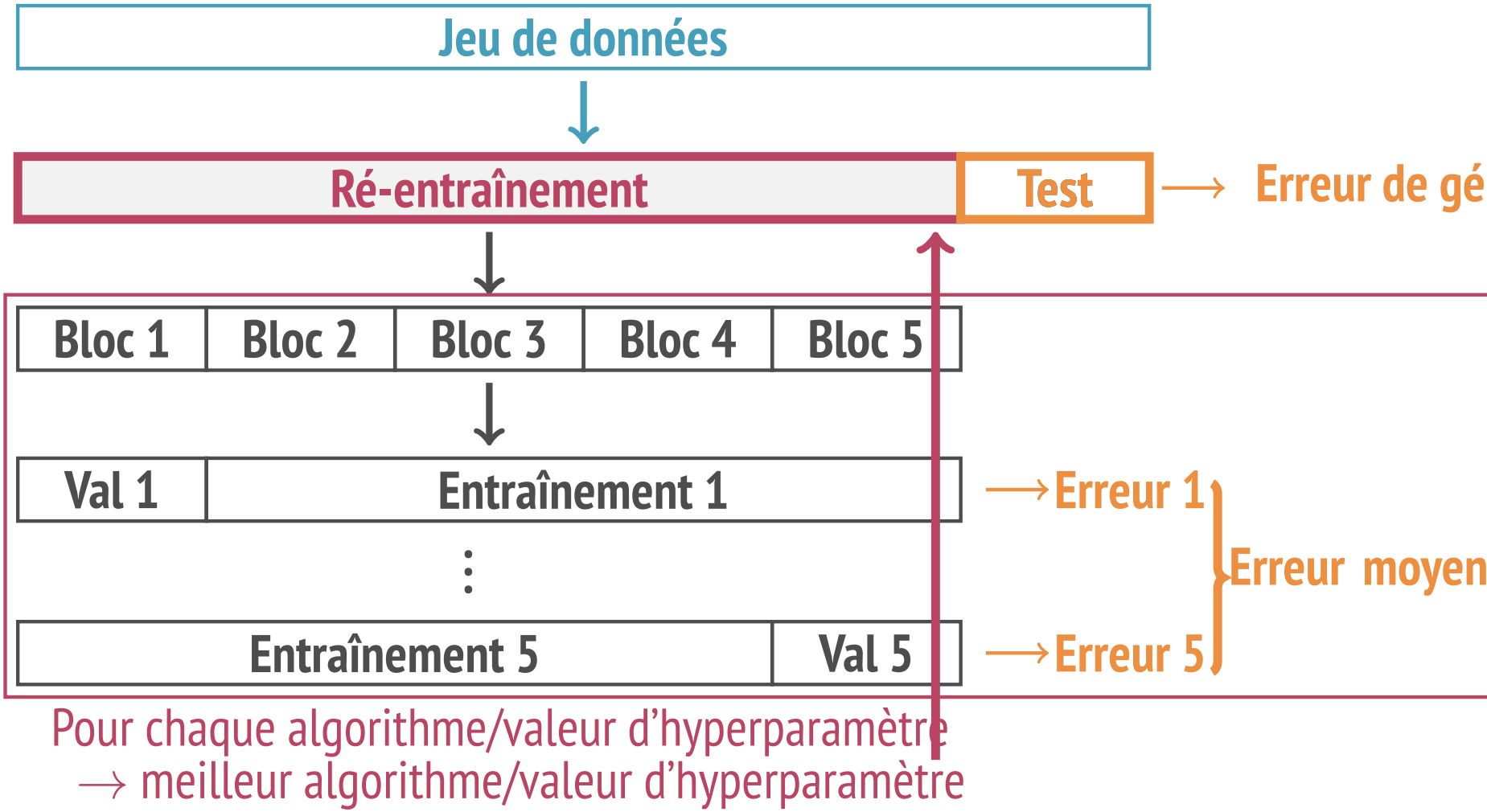
5. Sélection de modèle

Validation croisée (sélection de modèle)



5. Sélection de modèle

Validation croisée (sélection de modèle)



Références / page pub

- **Introduction au Machine Learning**, Chloé-Agathe Azencott, Dunod InfoSup
  Version électronique (PDF) sans exercices disponible gratuitement sur
  http://cazencott.info [lien cliquable]
- **Le Machine Learning avec scikit-learn**, Aurélien Géron, Dunod InfoSup
- **MOOC scikit-learn** [lien cliquable]